

40 ZETTABYTES

[43 TRILLION GIGABYTES]
of data will be created by
2020, an increase of 300
times from 2005



It's estimated that
2.5 QUINTILLION BYTES
[2.3 TRILLION GIGABYTES]
of data is created each day

400 MILLION TWEETS

are sent per day by
about 200 million
monthly active users



As of 2011, the global size
of data in healthcare was
estimated to be
150 EXABYTES
[161 BILLION GIGABYTES]



4 BILLION+
HOURS OF VIDEO
are watched on
YouTube each month
You Tube You Tube



1 IN 3 BUSINESS
don't trust the information
they use to make decisions



Poor data quality costs
the US economy around
\$3.1 TRILLION A YEAR

Volume

SCALE OF DATA

Variety

DIFFERENT
FORMS OF DATA

BIG DATA

Velocity

ANALYSIS OF STREAMING
DATA

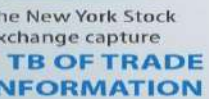
Veracity

UNCERTAINTY OF DATA

**27% OF
RESPONDENTS**

in one survey were unsure of
how much of their data was
inaccurate

By 2016, it is projected
there will be
**18.9 BILLION
NETWORK
CONNECTIONS** – almost 2.5 connections
per person on earth



The New York Stock
Exchange capture
**1 TB OF TRADE
INFORMATION**
during each trading session

100 SENSORS

Modern cars have close to
that monitor items such as
fuel level and tire pressure



Most companies in the
U.S. have at least
100 TERABYTES
[1,00,000 GIGABYTES]
of data stored



WORLD POPULATION:
7 BILLION



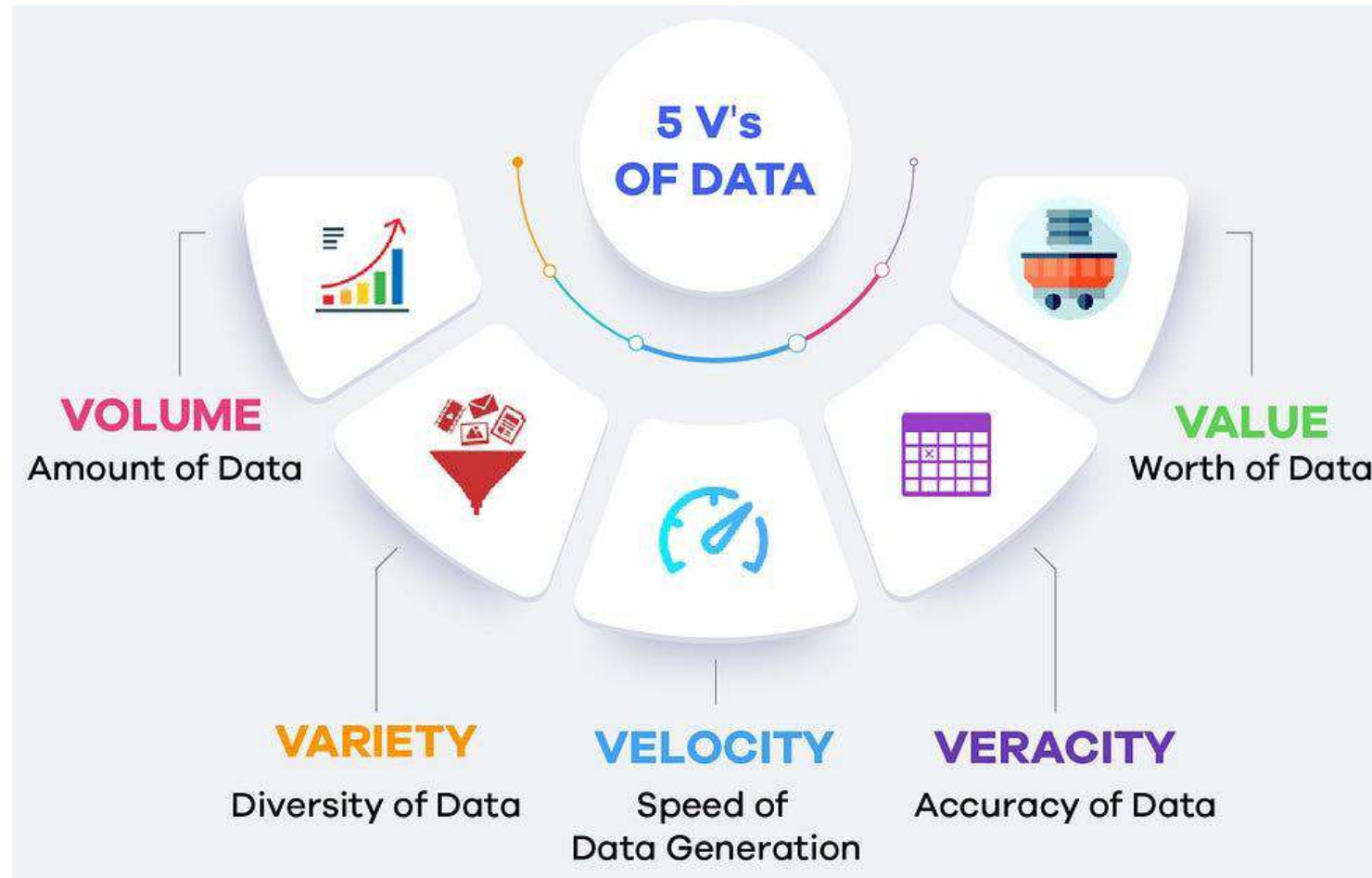
**6 BILLION
PEOPLE**
have cell phones

What is Big Data?

- Big Data refers to vast and rapidly growing volumes of data that are too large and complex for traditional data processing tools to manage. This data comes in many forms structured (e.g., tables), semi-structured (e.g., JSON, XML), and unstructured (e.g., text, images, video).
- The amount and availability of data is growing rapidly, spurred on by digital technology advancements, such as connectivity, mobility, the Internet of Things (IoT), and artificial intelligence (AI).
- As data continues to expand and proliferate, new big data tools are emerging to help companies collect, process, and analyze data at the speed needed to gain the most value from it.



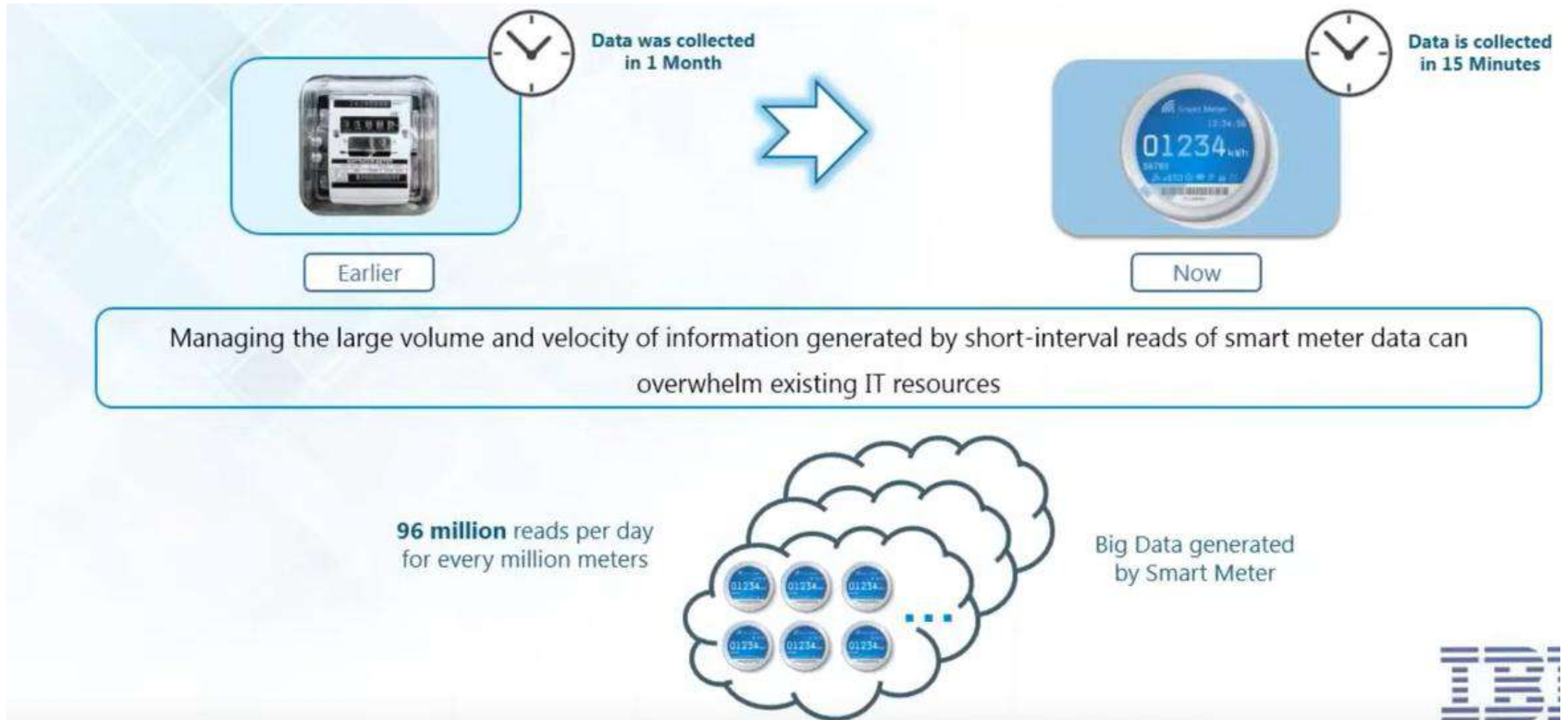
V's of Big Data



Additional V's:

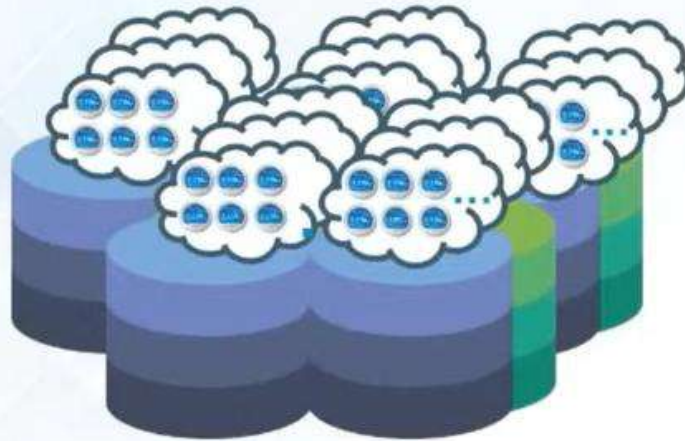
- ❑ **Variability:** Data meaning may change over time or context.
- ❑ **Visualization:** Making complex data understandable through visual tools (charts, graphs, dashboards).

IBM Big Data Analytics- Case Study



IBM Smart Meter

To manage and use this information to gain insight, utility companies must be capable of high-volume data management and advanced analytics designed to transform data into actionable insights.



Store

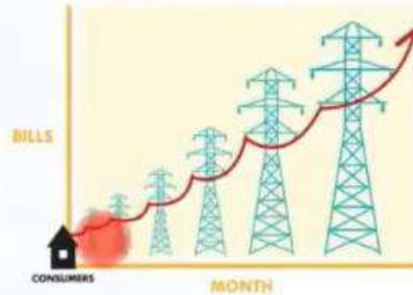


Analyze



Smart Meter Big Data Analytics

Before analyzing Big Data



Energy utilization and billing has increased

After analyzing Big Data



During peak-load the users require more energy



During off-peak times the users required less energy

Time-of-use pricing encourages cost-savvy retail like industrial heavy machines to be used at off-peak times

Solution Proposed

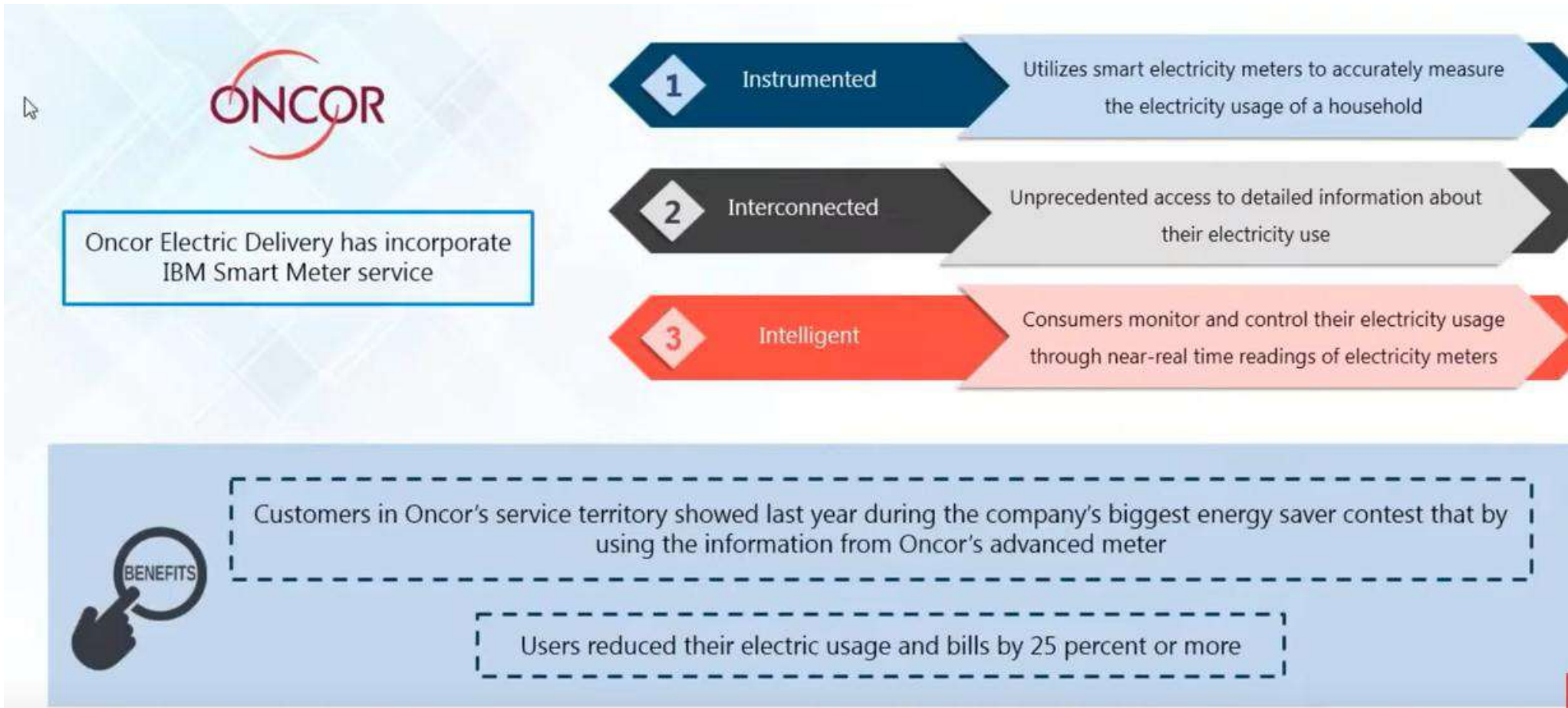
IBM offers an integrated suite of products designed to enable IT to leverage big data in a variety of ways that can contribute to the success of energy companies



IBM Solution



ONCOR using IBM Smart Meters



Big Data Management

- Big data management is the systematic process of data collection, data processing and data analysis that organizations use to transform raw data into actionable insights.
- The data flows into structured storage and processing environments, data integration tools can also help unify datasets from different sources, creating a single, comprehensive view that supports analysis.
- This stage also involves capturing metadata—information about the data's origin, format and other characteristics. Metadata can provide essential context for future organizing and processing data down the line.
- Large datasets can be prone to errors and inaccuracies that might affect the reliability of future insights. Validation and cleansing procedures, such as schema validation and deduplication, can help to address errors, resolve inconsistencies and fill in missing information.

Problems with Big Data

Problem 1: Storing exponentially growing huge datasets

- Data generated in past **2 years** is more than the previous history in total
- By 2020, total digital data will grow to **44 Zettabytes** approximately
- By 2020, about **1.7 MB** of new info will be created every second for every person



Problems with Big Data

Problem 2: Processing data having complex structure

Structured

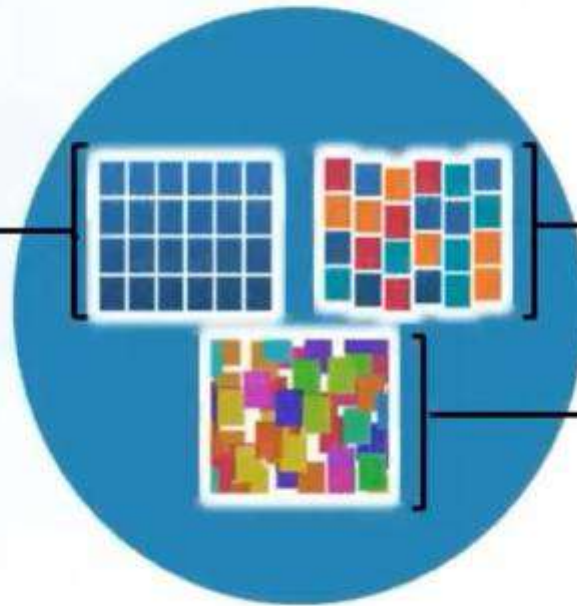
- Organized data format
- Data schema is fixed
- Ex: RDBMS data, etc.

Semi – Structured

- Partial organized data
- Lacks formal structure of a data model
- Ex: XML & JSON files, etc.

Unstructured

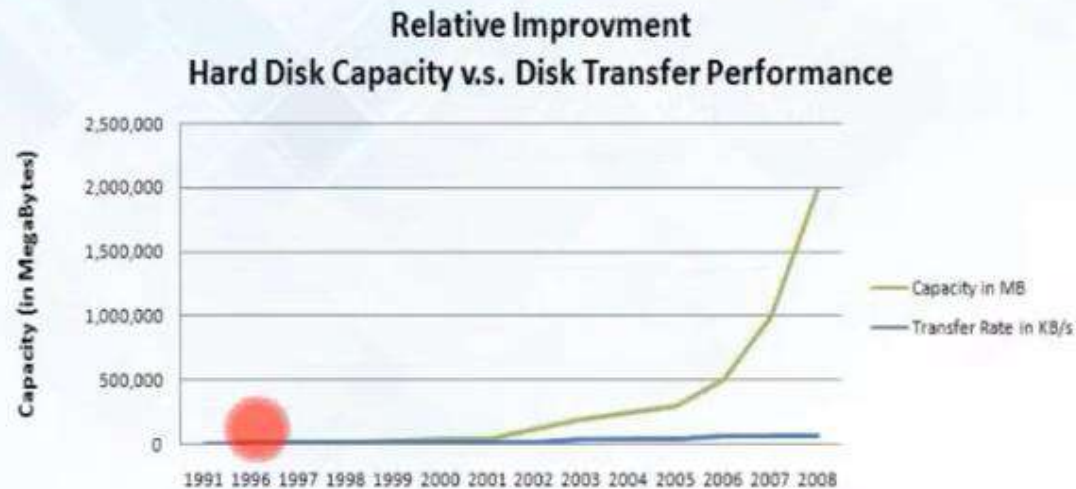
- Un-organized data
- Unknown schema
- Ex: multi-media files, etc.



Problems with Big Data

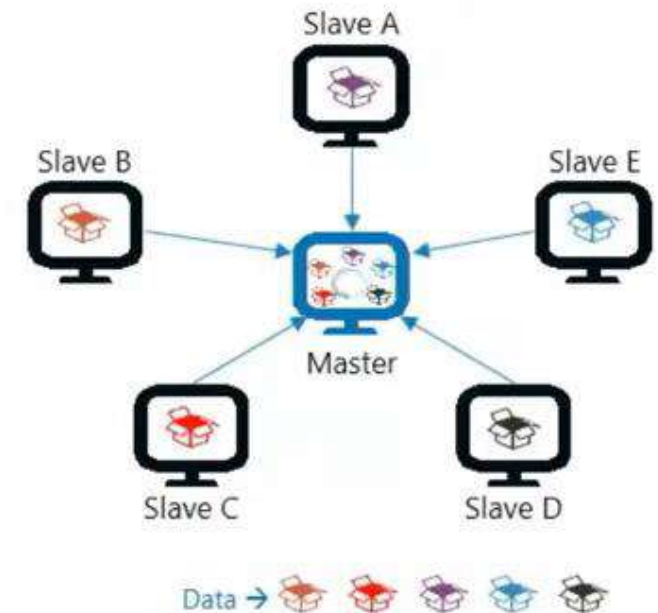
Problem 3: Processing data faster

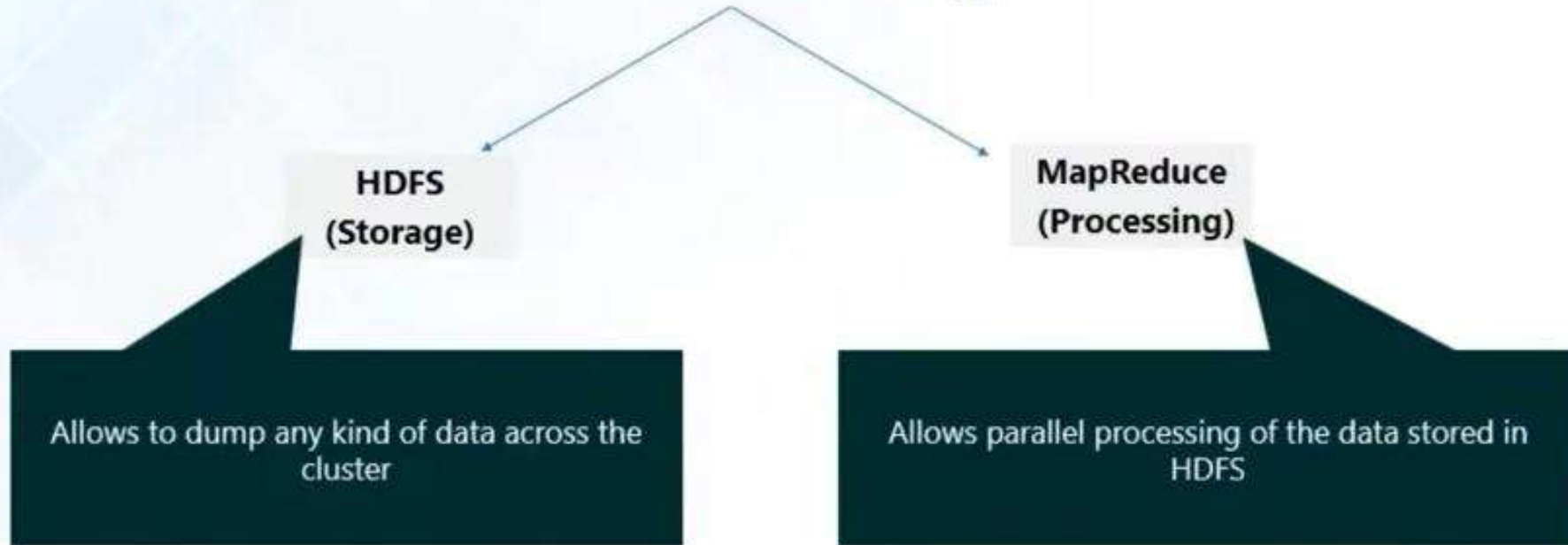
The data is growing at much faster rate than that of disk read/write speed



Source: Tom's Hardware

Bringing huge amount of data to computation unit becomes a bottleneck





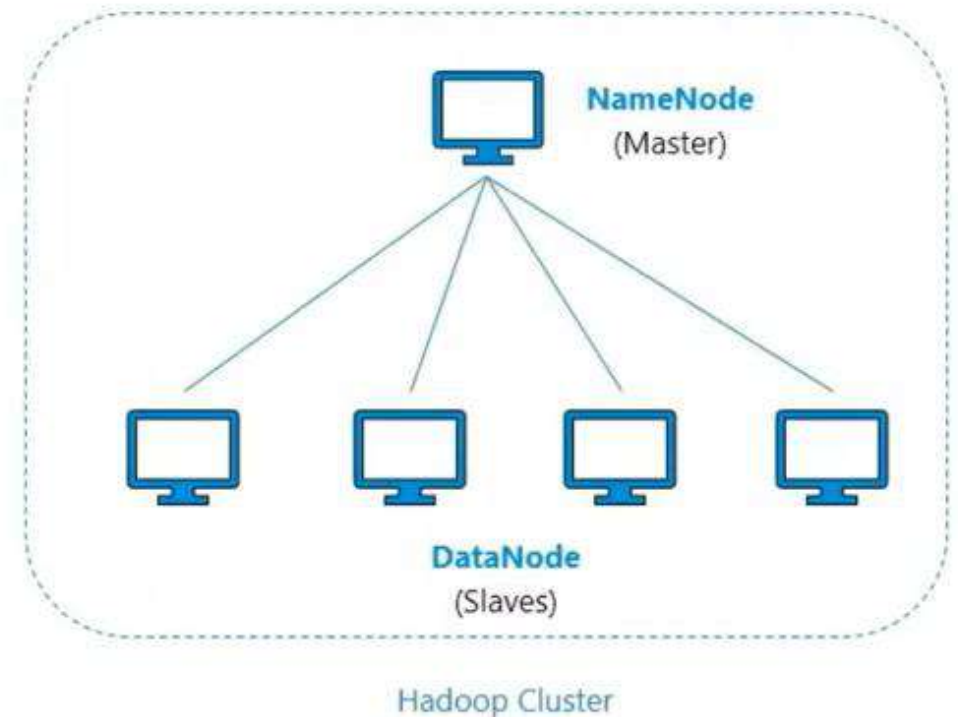
Hadoop-as-a-Solution

Hadoop Distributed File System (HDFS)

HDFS creates a level of abstraction over the resources, from where we can see the whole HDFS as a single unit.

HDFS has two core components, i.e. NameNode and DataNode.

- The *NameNode* is the main node that contains metadata about the data stored.
- Data is stored on the *DataNodes* which are commodity hardware in the distributed environment.

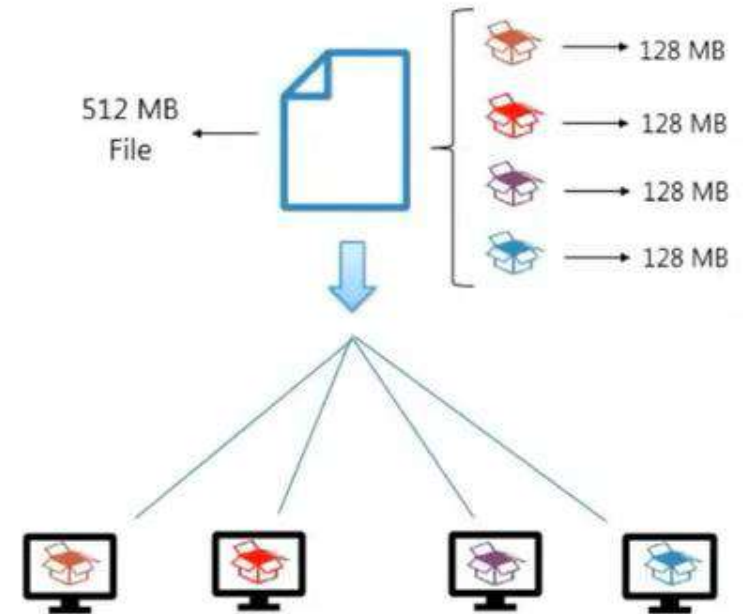


Solution to Problem 1

Problem 1: Storing exponentially growing huge datasets

Solution: HDFS

- Storage unit of Hadoop
- It is a Distributed File System
- Divide files (input data) into smaller chunks and stores it across the cluster
- Scalable as per requirement

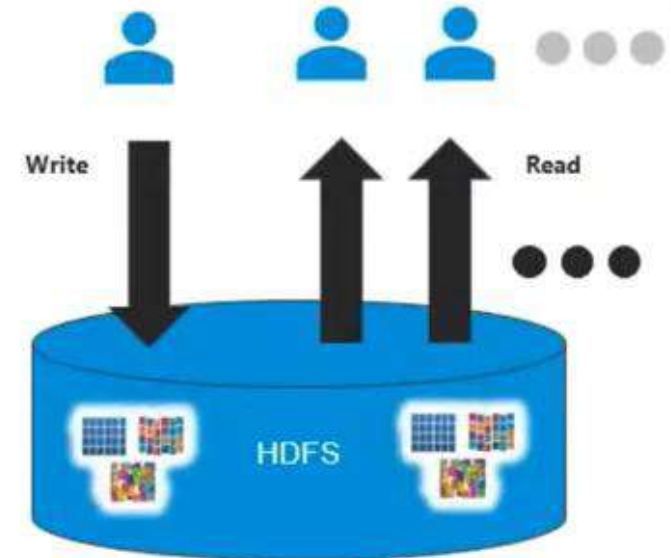


Solution to Problem 2

Problem 2: Storing unstructured data

Solution: HDFS

- Allows to store any kind of data, be it structured, semi-structured or unstructured
- Follows WORM (Write Once Read Many)
- No schema validation is done while dumping data



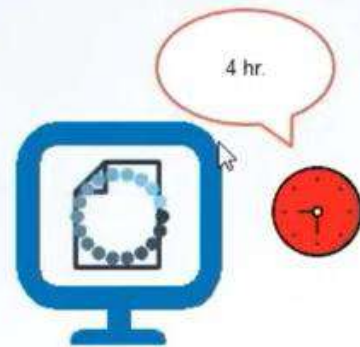
Solution to Problem 3

Problem 3: Processing data faster

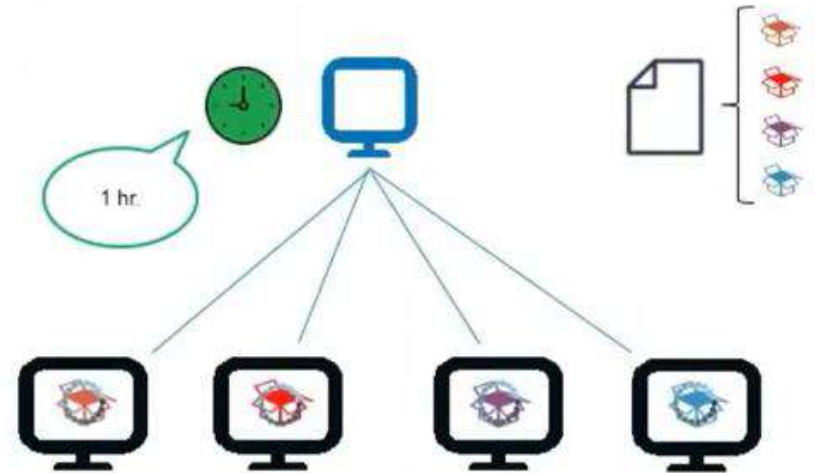
Solution: Hadoop MapReduce

- Provides parallel processing of data present in HDFS
- Allows to process data locally i.e. each node works with a part of data which is stored on it

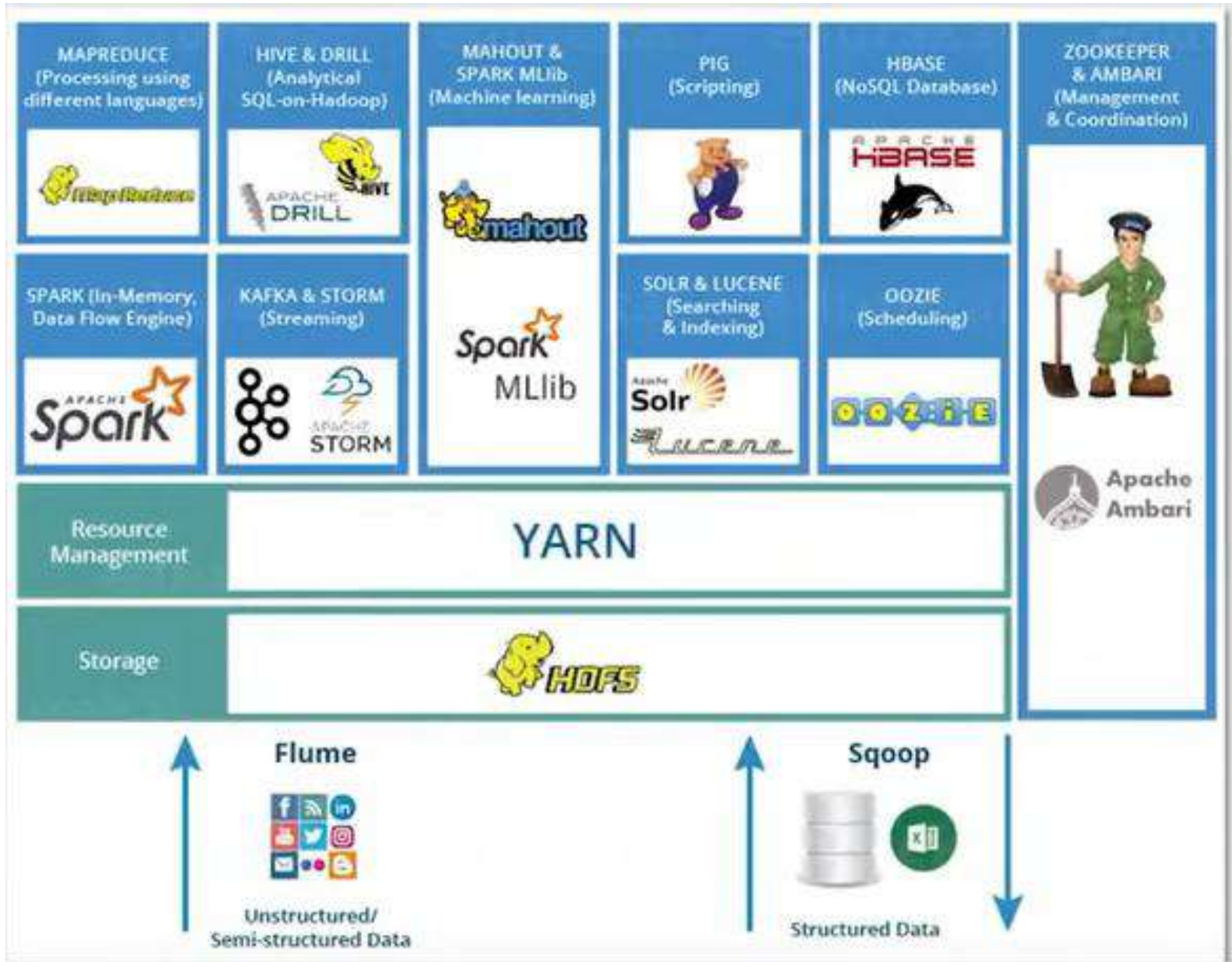
1



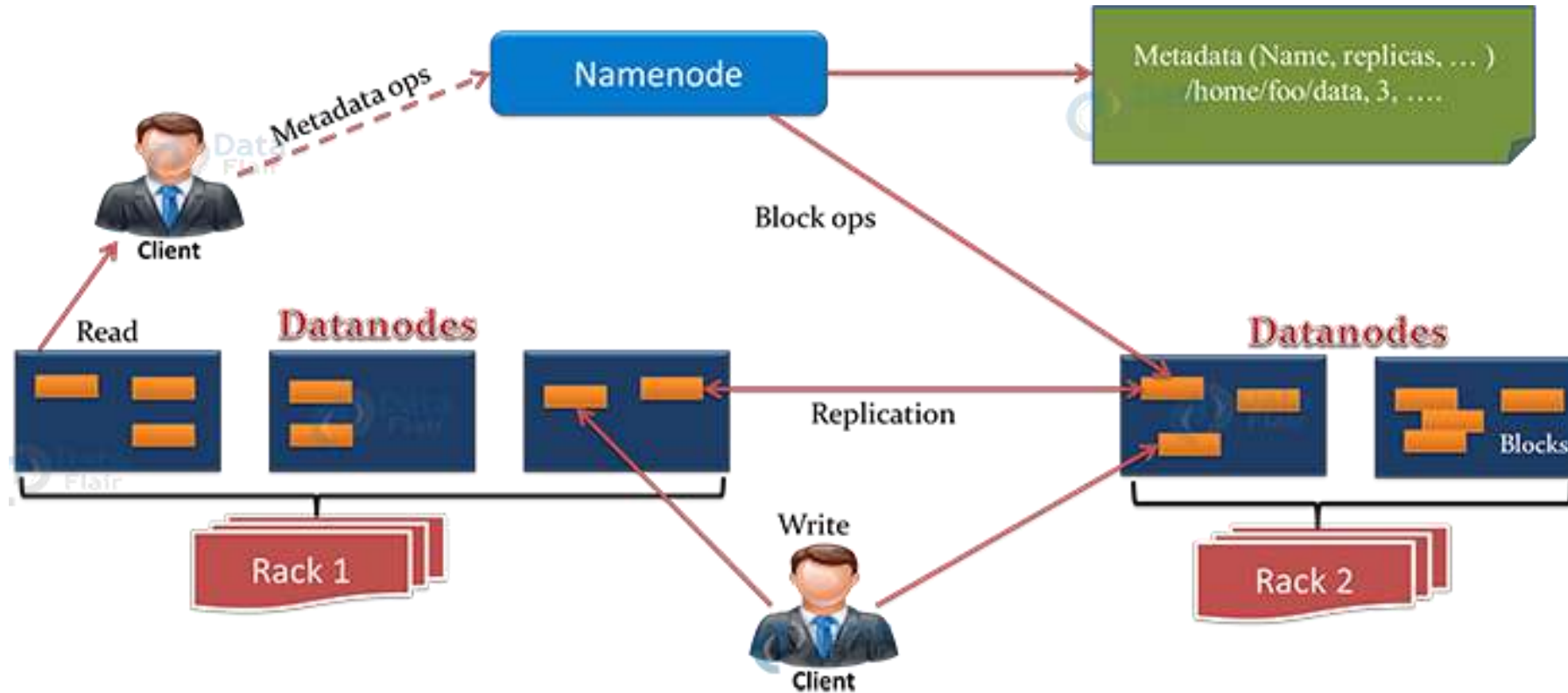
2



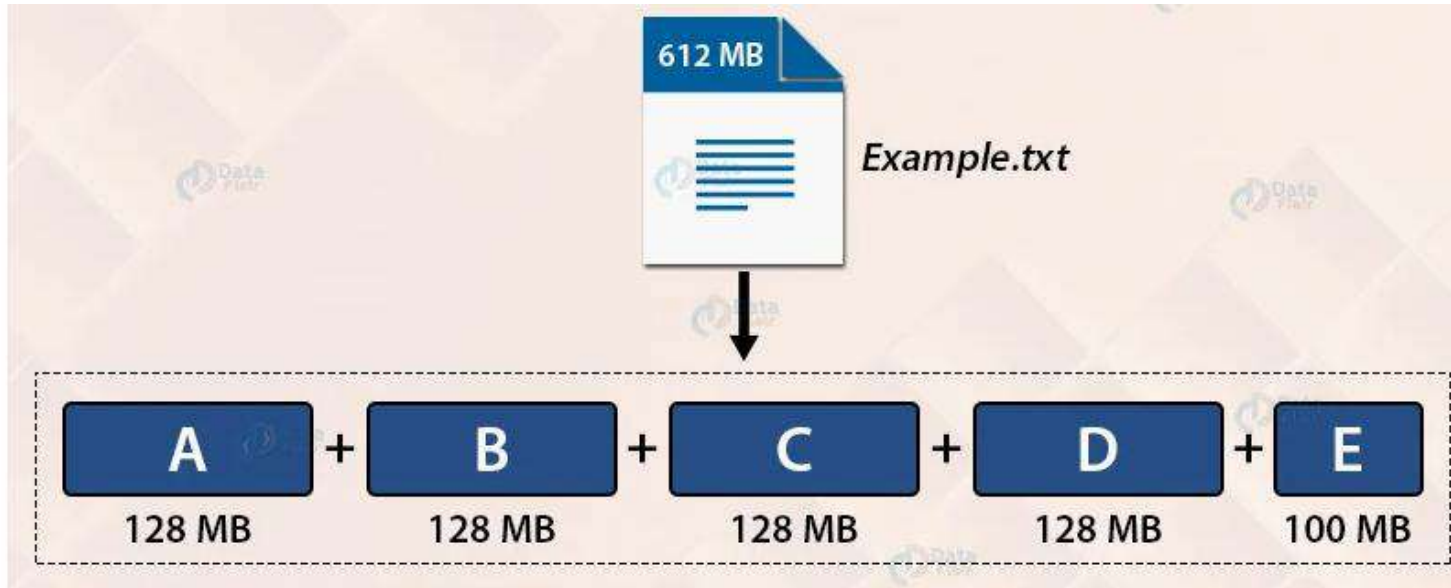
Big Data Frameworks



HDFS Architecture



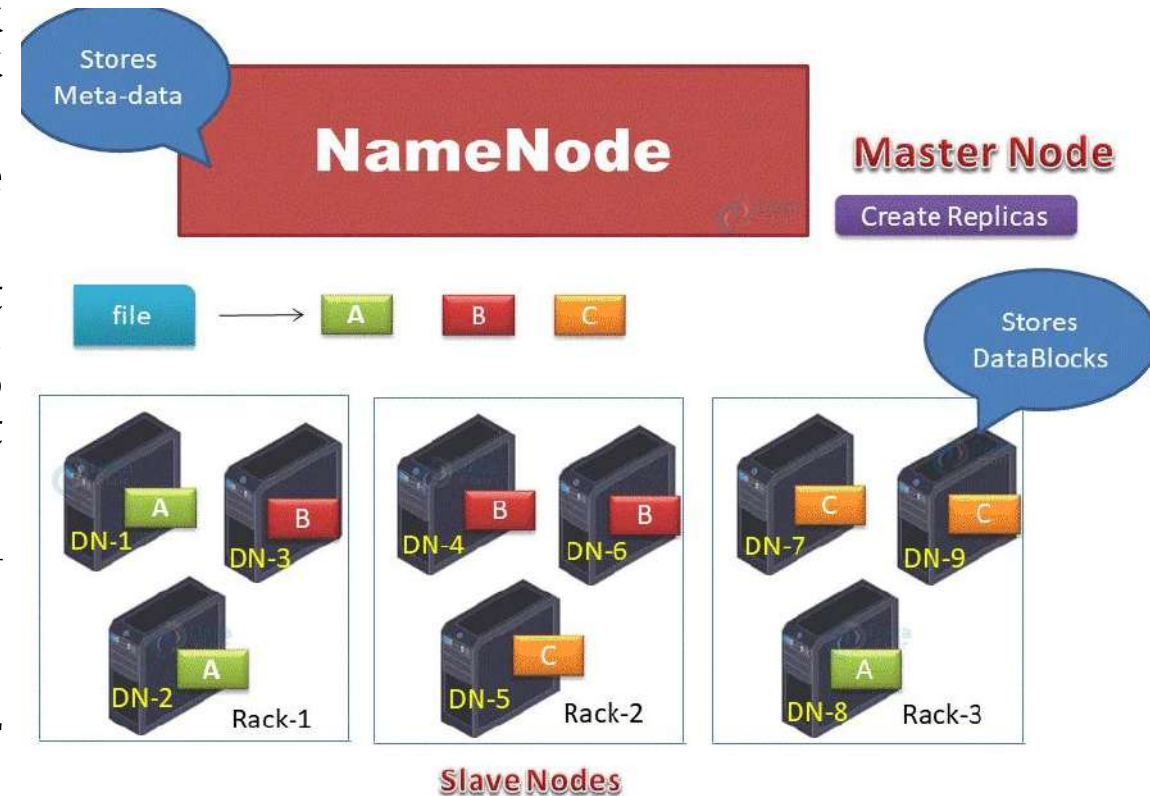
- ❑ Hadoop Distributed File System follows the **master-slave architecture**. Each cluster comprises a **single master node** and **multiple slave nodes**. Internally the files get divided into one or more **blocks**, and each block is stored on different slave machines depending on the **replication factor**



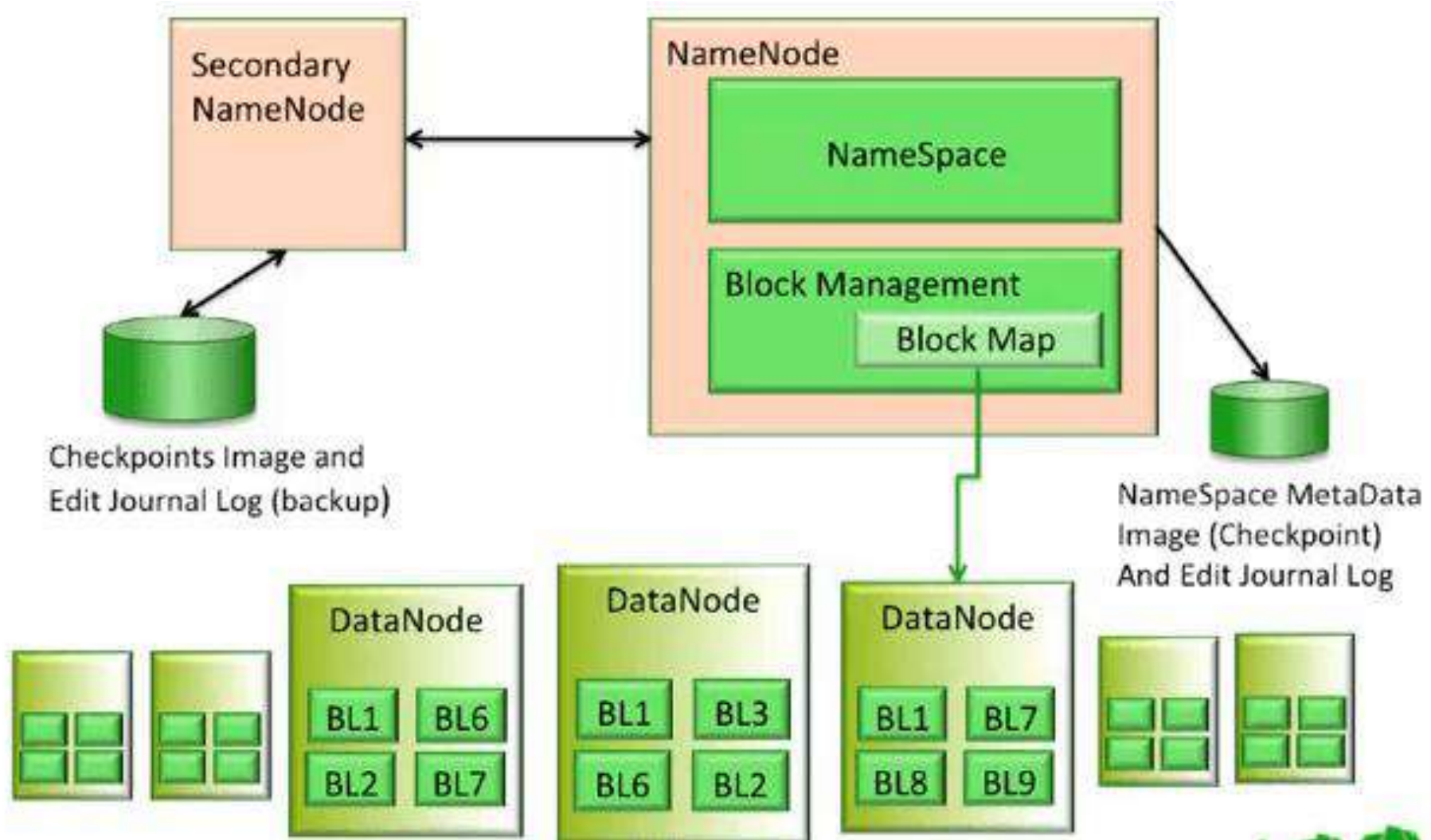
- ❑ Internally, HDFS split the file into block-sized chunks called a block. The size of the block is **128 Mb** by default. One can configure the block size as per the requirement.
- ❑ For example, if there is a file of size 612 Mb, then HDFS will create four blocks of size 128 Mb and one block of size 100 Mb.

Rack Awareness in HDFS Architecture

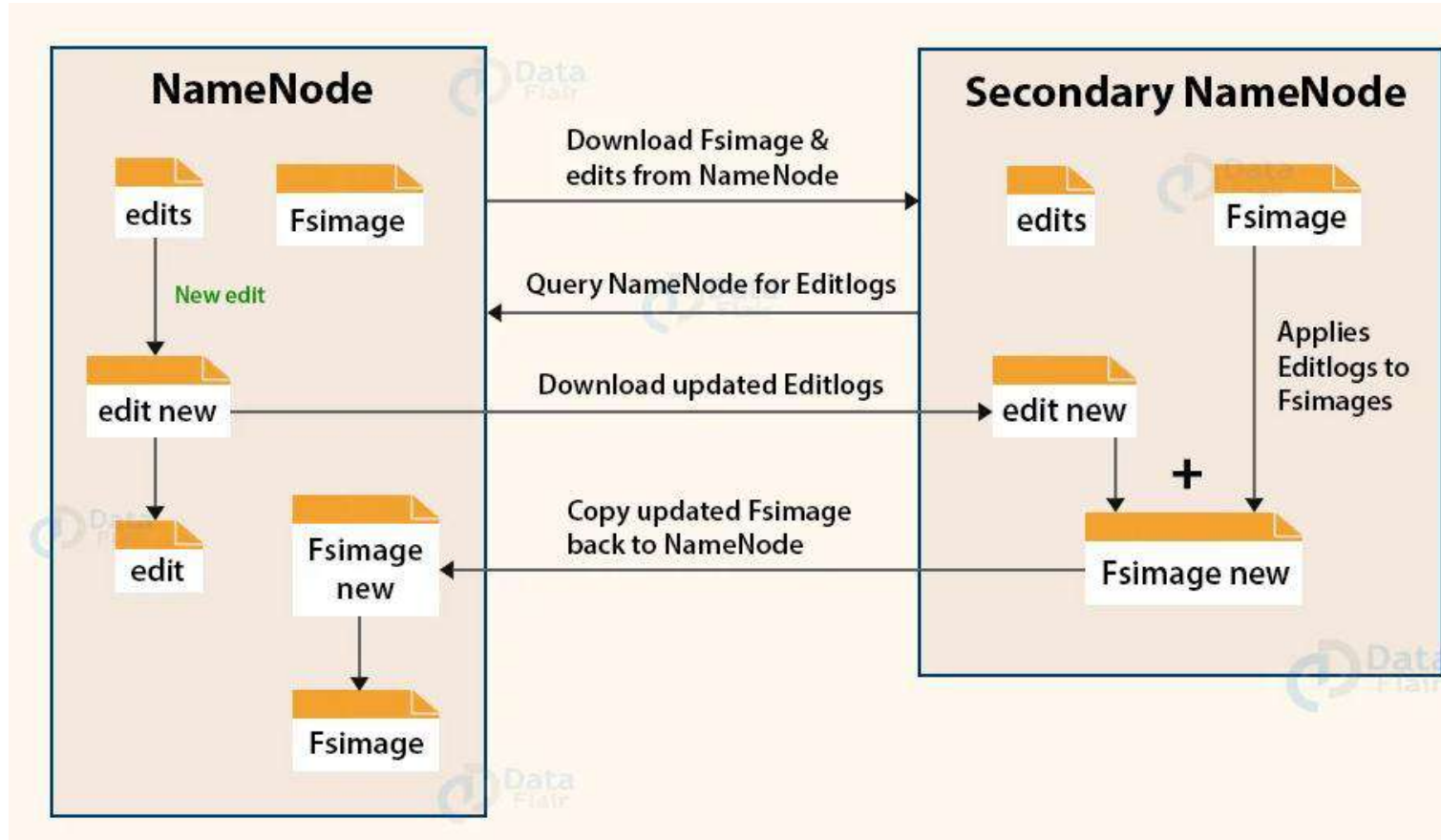
- **Rack** is the collection of around 40-50 machines (DataNodes) connected using the same network switch. If the network goes down, the whole rack will be unavailable.
- **Rack Awareness** is the concept of choosing the closest node based on the rack information.
- To ensure that all the replicas of a block are not stored on the same rack or a single rack, NameNode follows a rack awareness algorithm to store replicas and provide latency and fault tolerance.
- Suppose if the replication factor is 3, then according to the rack awareness algorithm:
 - The first replica will get stored on the local rack.
 - The second replica will get stored on the other DataNode in the same rack.
 - The third replica will get stored on a different rack.



HDFS Architecture



Secondary NameNode



- It groups the Edit logs and Fsimage from NameNode together.
- It continuously reads the MetaData from the RAM of NameNode and writes into the Hard Disk.



Secondary NameNode (Checkpoint Node)

- The Checkpoint node is a node that periodically creates checkpoints of the namespace.
- Checkpoint Node in Hadoop first downloads Fsimage and edits from the Active Namenode. Then it merges them (Fsimage and edits) locally, and at last, it uploads the new image back to the active NameNode.
- It stores the latest checkpoint in a directory that has the same structure as the Namenode's directory. This permits the checkpointed image to be always available for reading by the NameNode if necessary.

Responsibilities

- **Key Responsibilities of NameNode:**

- Maintaining the filesystem [tree](#) and metadata.
- Managing the mapping of file blocks to DataNodes.
- Ensuring data integrity and coordinating replication of data blocks.

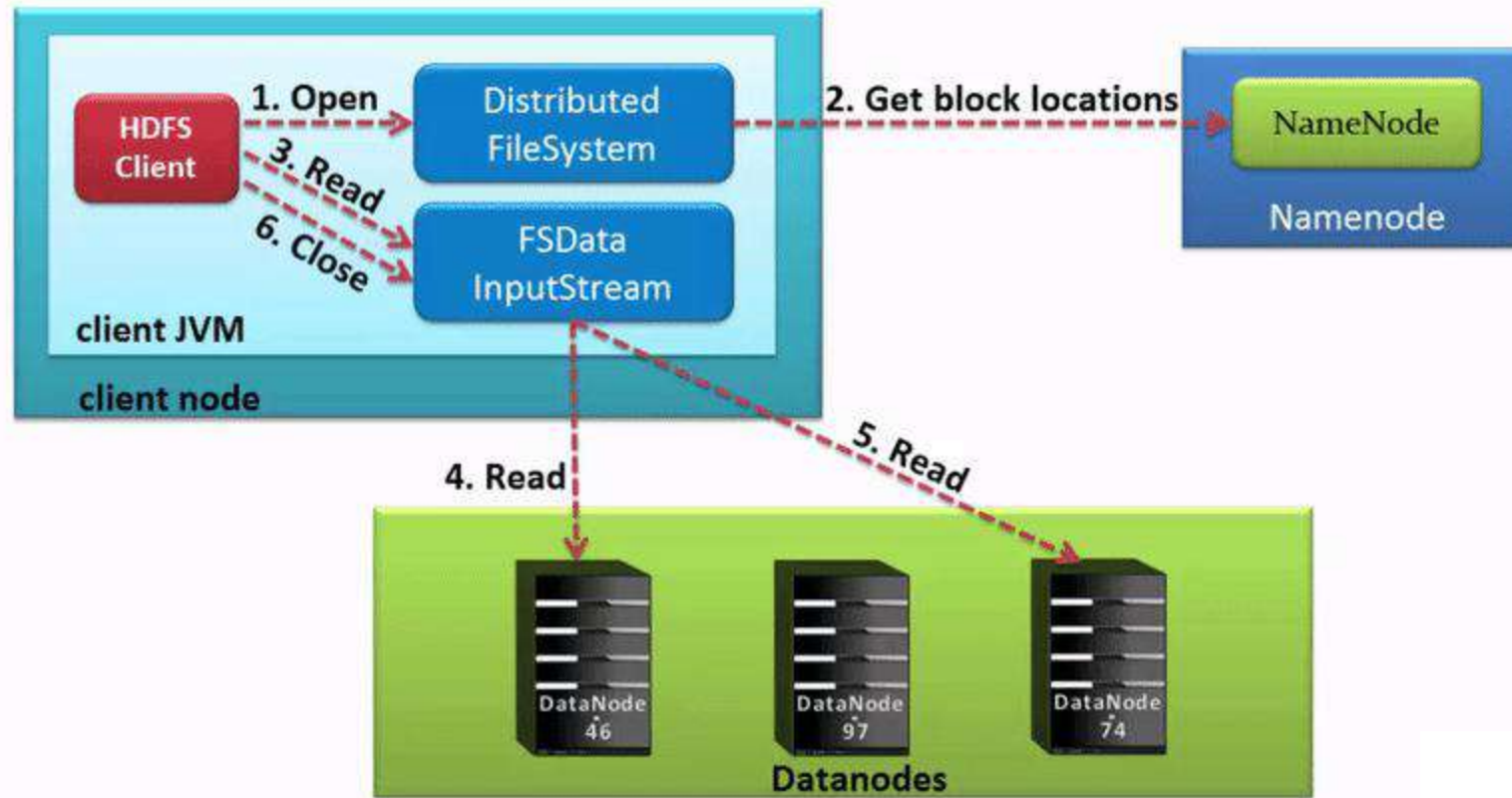
Key Responsibilities of DataNode:

- Storing data blocks and serving read/write requests from clients.
- Performing block creation, deletion, and replication upon instruction from the NameNode.
- Periodically sending block reports and heartbeats to the NameNode to confirm its status.

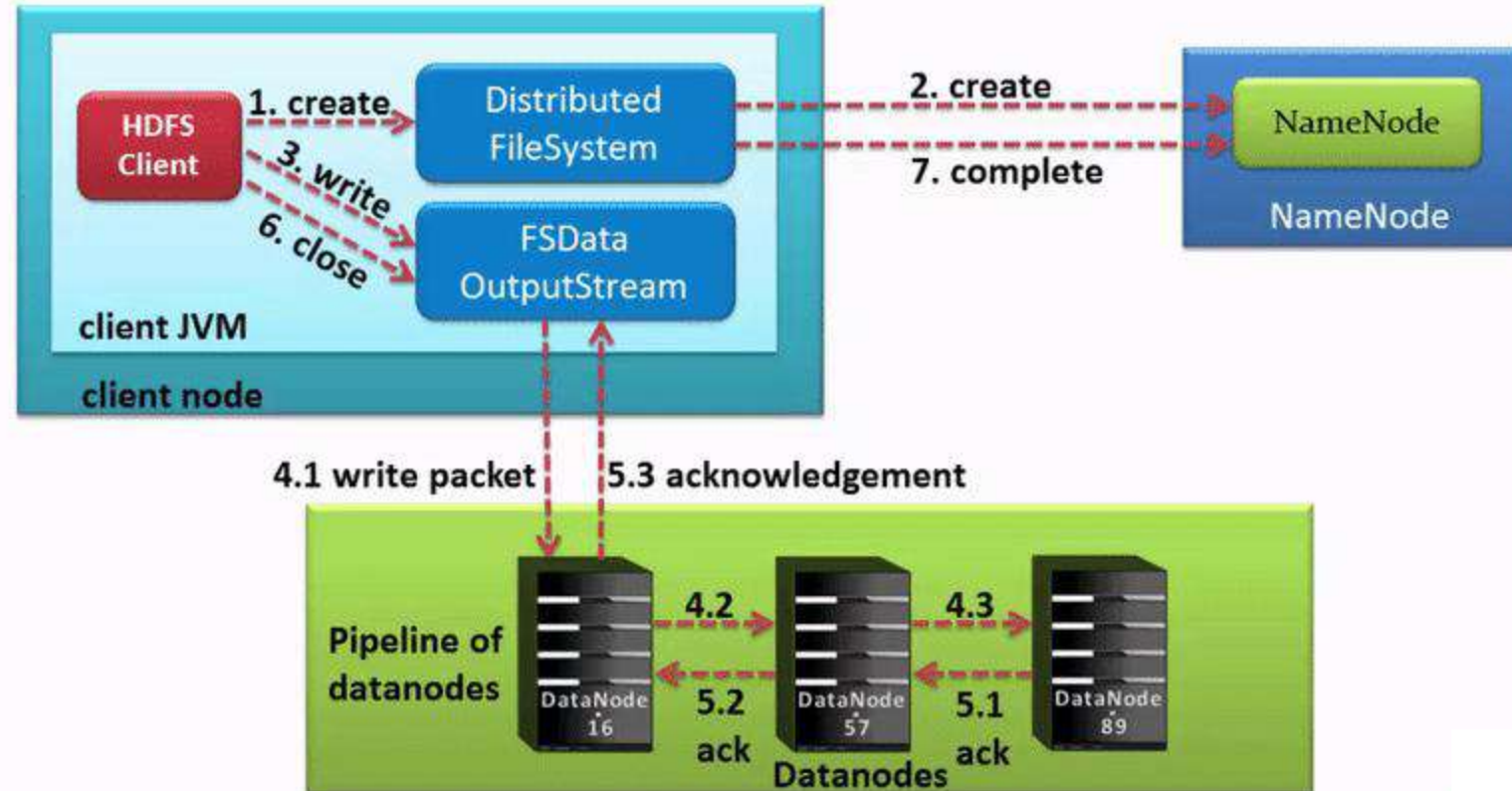
Key Responsibilities of Secondary NameNode:

- Merging EditLogs with FsImage to create a new checkpoint.
- Helping to manage the NameNode's namespace metadata.

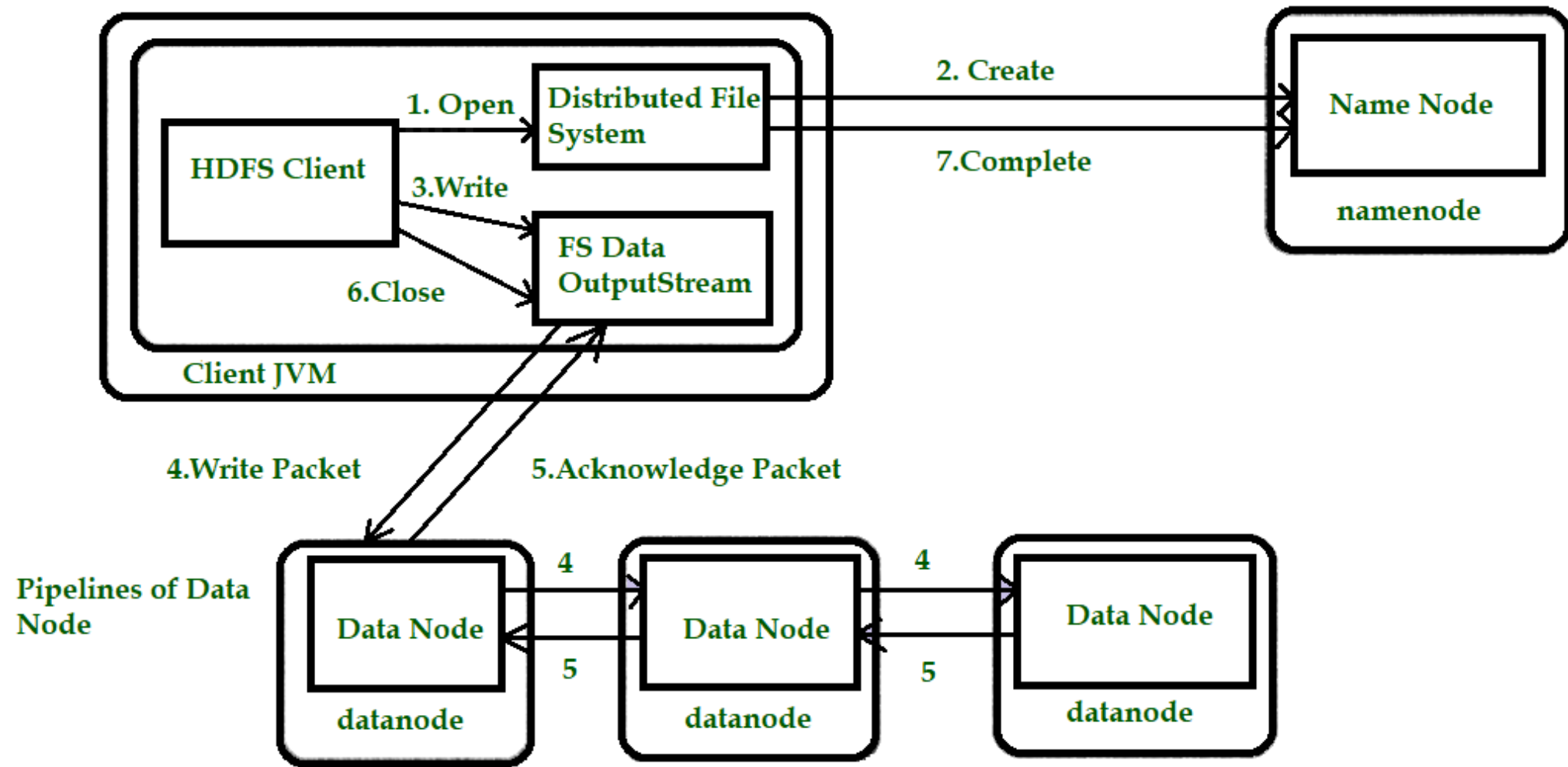
Hadoop HDFS Read Operation



Hadoop HDFS Write Operation



Hadoop HDFS Write Operation



Hadoop HDFS Write Operation

Step 1: The client creates the file by calling `create()` on `DistributedFileSystem(DFS)`.

Step 2: DFS makes an RPC call to the name node to create a new file in the file system's namespace, with no blocks associated with it. The name node performs various checks to make sure the file doesn't already exist and that the client has the right permissions to create the file. If these checks pass, the name node prepares a record of the new file; otherwise, the file can't be created and therefore the client is thrown an error i.e. `IOException`. **The DFS returns an `FSDataOutputStream` for the client to start out writing data to.**

Step 3: Because the client writes data, the `DFSOutputStream` splits it into packets, which it writes to an indoor queue called the info queue.

The `DataStreamer` streams the packets to the primary data node within the pipeline, which stores each packet and forwards it to the second data node within the pipeline.

Step 4: Similarly, the second data node stores the packet and forwards it to the third (and last) data node in the pipeline.

Step 5: The `DFSOutputStream` sustains an internal queue of packets that are waiting to be acknowledged by data nodes, called an "ack queue".

Step 6: This action sends up all the remaining packets to the data node pipeline and waits for acknowledgments before connecting to the name node to signal whether the file is complete or not.